

Page Ranking

Procheta Sen



How does Google decide which pages to show first?

The screenshot shows a Google search interface with the query 'panda'. The search results are as follows:

Google panda

Search filters: All, Images, Videos, Shopping, News, More, Settings, Tools

About 610,000,000 results (0.70 seconds)

Ad · <https://www.pandasecurity.com/> · 0808 169 2281

50% Off Panda™ Antivirus 2021 · VPN Included at No Extra Cost

Next-gen Antivirus that Blocks 100% of Online Threats. Install Now & Get a Massive 50% Off. Choose from 4 Plans Depending On Your Needs. Works On All Devices & Operating...

Spring Sale: 50% off Panda Antivirus 2021 · Valid Apr 1 - Apr 30

Protection Advanced -40% The Most Advanced Protection for Your Digital Life, Data & Identity	Protection Premium -50% VPN & Maximum Protection with Premium Services & 24-7 VIP Support
--	---

https://en.wikipedia.org/wiki/Giant_panda

Giant panda - Wikipedia

The giant panda (*Ailuropoda melanoleuca*; Chinese: 大熊猫; pinyin: dàxióngmāo), also known as the panda bear or simply the panda, is a bear native to South Central China. It is characterised by its bold black-and-white coat and rotund body.

Species: *A. melanoleuca* Genus: *Ailuropoda*
Family: *Ursidae* Kingdom: *Animalia*

[Red panda](#) · [Qinling panda](#) · [Panda \(disambiguation\)](#) · [Panda tea](#)

https://en.wikipedia.org/wiki/Google_Panda

Google Panda - Wikipedia

Google Panda is a major change to Google's search results ranking algorithm that was first released in February 2011. The change aimed to lower the rank of ...

<https://www.worldwildlife.org/species/giant-panda>

Giant Panda | Species | WWF

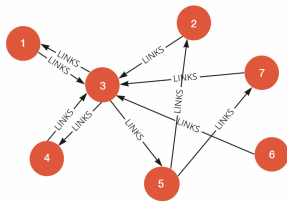
Pandas live mainly in temperate forests high in the mountains of southwest China, where they subsist almost entirely on bamboo. They must eat around 26 to 84 ...

PageRank algorithm

- ▶ Proposed in 1998 by Sergey Brin and Larry Page (the founders of Google).
- ▶ PageRank (PR) is an algorithm used by Google Search to rank web pages in their search engine results.
- ▶ Algorithm uses the web graph to compute a measure of importance (PageRank) of every webpage.
- ▶ PageRank is a static ranking of Web pages, i.e. a PageRank value is computed for each page off-line and it does not depend on search queries.

Web Graph

- ▶ Nodes: webpages
- ▶ Directed edges: hyperlinks between the pages



<https://networkx.guide/algorithms/link-analysis/pagerank/>

Figure:

Terminology

- ▶ **In-links** of page a : the hyperlinks that point to page a from other pages. Usually, hyperlinks from the same page are ignored.
- ▶ **Out-links** of page a : the hyperlinks that point out to other pages from page a . Usually, hyperlinks to the same site are ignored.

PageRank algorithm idea

- ▶ The PageRank score of each page can be regarded as its prestige.
- ▶ PageRank interprets a hyperlink from page a to page b as a vote, by page a , for page b .
- ▶ However, unlike Degree Prestige, PageRank looks at more than just the sheer number of votes or links that a page receives. It also analyzes the page that casts the vote.
- ▶ Votes casted by pages that are themselves “important” are weighted more heavily and help to make other pages more “important”.

PageRank algorithm idea

- ▶ The importance of page a (i.e. a 's PageRank score) is determined by summing up the PageRank scores of all pages that point to a .
- ▶ Since a page may point to many other pages, its PageRank score should be shared among all the pages that it points to.

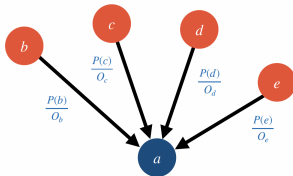
PageRank algorithm idea

Notation

- ▶ $P(a)$ – PageRank score of page a
- ▶ O_a – the number of out-links of page a
- ▶ E – the set of arcs (directed edges) of the graph

The PageRank score of page a is defined by

$$P(a) = \sum_{(x,a) \in E} \frac{P(x)}{O_x}$$



PageRank algorithm idea

- ▶ Let $1, 2, \dots, n$ be the vertices of the graph.
- ▶ Let $\bar{P}$ be a n -dimensional column vector of PageRank scores of the vertices, i.e.

$$\bar{P} = (P(1), P(2), \dots, P(n))^T$$

- ▶ Let $\bar{A}$ be the modified adjacency matrix of our graph with

$$\bar{A}_{ij} = \begin{cases} \frac{1}{O_i}, & \text{if } (i, j) \in E \\ 0, & \text{otherwise} \end{cases}$$

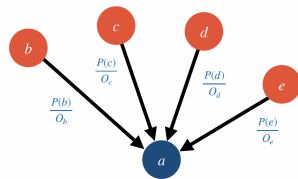


Figure:

PageRank algorithm idea

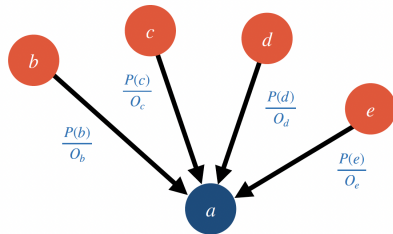
Under this notation we can write the system

$$P(i) = \sum_{(j,i) \in E} \frac{P(j)}{O_j}, \quad i = 1, \dots, n$$

of n equations in the matrix form as follows

$$\bar{P} = \bar{A}^T \bar{P}$$

The solution $\bar{P}$ is an **eigenvector** corresponding to eigenvalue of 1.



PageRank algorithm idea

$$\bar{P} = \bar{A}^T \bar{P}$$

The solution $\bar{P}$ is an **eigenvector** corresponding to eigenvalue of 1.

If $\bar{A}$ satisfies certain conditions, then 1 is the largest eigenvalue of $\bar{A}$ and the solution $\bar{P}$ can be found by the **power iteration algorithm**:

Power Iteration

- ▶ Starting with $\bar{P}_0$
- ▶ Iteratively compute

$$\bar{P}_i = \bar{A} \bar{P}_{i-1}$$

- ▶ Until

$$\|\bar{P}_i - \bar{P}_{i-1}\|_1 \leq \epsilon$$

where ϵ is a pre-specified threshold

PageRank algorithm idea

Unfortunately, the **conditions** are not satisfied for the real Web graph.

Page Ranking Algorithm

Procheta Sen



PageRank algorithm: Markov chain perspective

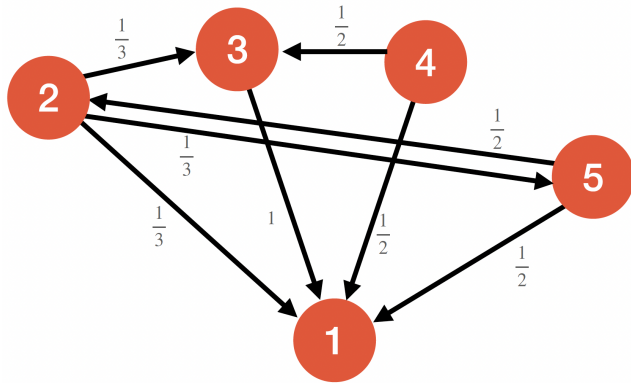
In the Markov chain model:

- ▶ each **node** (webpage) of the graph is regarded as a **state**
- ▶ an **arcs** (hyperlink) is a **transition**, which leads from one state to another state
- ▶ at a fixed state all transitions are equally probable

This framework models a **random Web surfer**:

- ▶ an imaginary surfer is randomly clicking on links
- ▶ all links have an equal probability of getting clicked
- ▶ “back” button on the browser is not used and the surfer does not type in an URL
- ▶ after the surfer made many clicks, what is the probability that they end up on a particular webpage?

PageRank algorithm: Markov chain perspective



For vertex x , each transition from x to one of its neighbours is $\frac{1}{O_x}$

PageRank algorithm: Markov chain perspective

- ▶ Let $\bar{A}$ be the **state transition probability matrix**:

$\bar{A}_{ij}$ is equal to the transition probability that the surfer in state (page) i will move to state j , i.e.

$$\bar{A}_{ij} = \frac{1}{O_i}, \text{ if } (i, j) \in E, \text{ and}$$
$$\bar{A}_{ij} = 0, \text{ otherwise}$$

- ▶ Let $\bar{P}_0 = (P_0(1), P_0(2), \dots, P_0(n))^T$ be an initial probability distribution vector that a surfer is at each state (page).

PageRank algorithm: Markov chain perspective

Then we have

$$\sum_{i=1}^n P_0(i) = 1 \quad \text{— initially the surfer is in one of the states (pages)}$$

$$\sum_{j=1}^n \bar{A}_{ij} = 1 \quad \text{— the surfer always makes a click when in state } i$$

The latter equation is not quite true for some Web pages because they have no out-links.

Such pages (nodes) are called **dangling pages** (nodes).

If matrix $\bar{A}$ satisfies that equation for every $i = 1, \dots, n$, we say that $\bar{A}$ is the **stochastic matrix** of Markov chain.

PageRank algorithm: Markov chain perspective

Assume for the time being that $\bar{A}$ is a stochastic matrix.

A standard question about Markov chains:

Given the initial probability distribution $\bar{P}_0$ at the beginning, what is the probability that m steps/ transitions later the Markov chain (random surfer) will be at each state j ?

The probability that the random surfer is in state j after 1 transition can be computed as

$$P_1(j) = \sum_{i=1}^n \bar{A}_{ij}(1) P_0(i)$$

where $\bar{A}_{ij}(1)$ is the probability of going from i to j in 1 transition, and $\bar{A}_{ij}(1) = \bar{A}_{ij}$.

PageRank algorithm: Markov chain perspective

By the Ergodic Theorem of Markov chains,

a finite Markov chain defined by **the stochastic transition matrix** A has a unique **stationary probability distribution** if A is **irreducible** and **aperiodic**.

The stationary probability distribution means that after a series of transitions $\bar{P}_k$ will converge to a steady-state probability vector $\bar{\Pi}$ regardless of the choice of the initial probability vector $\bar{P}_0$, i.e.

$$\lim_{k \rightarrow \infty} \bar{P}_k = \bar{\Pi}$$

PageRank algorithm: Markov chain perspective

When we reach the steady-state, we have $\bar{P}_k = \bar{P}_{k-1} = \bar{\Pi}$, and thus $\bar{\Pi} = \bar{A}^T \bar{\Pi}$.

That is $\bar{\Pi}$ is the **principal** eigenvector of $\bar{A}^T$ with eigenvalue 1.

(Known fact: A **stochastic matrix** always has an eigenvalue 1. All other eigenvalues are in absolute value smaller or equal to 1)

If we treat $\bar{\Pi}$ as the PageRank vector $\bar{P}$ we obtain our previous equation

$$\bar{P} = \bar{A}^T \bar{P}$$

PageRank algorithm: Markov chain perspective

We want our matrix $\bar{A}$ to satisfy the following 3 conditions:

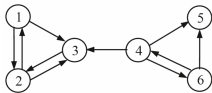
1. To be **stochastic**
2. To be **irreducible**
3. To be **aperiodic**

None of these conditions is satisfied for the real web graph.

However, we can modify matrix $\bar{A}$ so that it satisfies all three conditions!

Modification I: make $\bar{A}$ stochastic

For every dangling page i , add a complete set of outgoing links. The transition probability of going from i to every page is $1/n$



Hyperlink graph

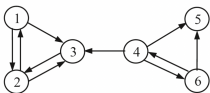
$$\bar{A} = \begin{pmatrix} 0 & 1/2 & 1/2 & 0 & 0 & 0 \\ 1/2 & 0 & 1/2 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1/3 & 0 & 1/3 & 1/3 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1/2 & 1/2 & 0 \end{pmatrix}$$

Transition probability matrix

5 is a dangling page, and hence $\bar{A}$ is **not** a stochastic matrix.

Modification I: make $\bar{A}$ stochastic

For every dangling page i , add a complete set of outgoing links. The transition probability of going from i to every page is $1/n$



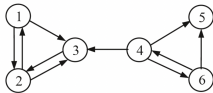
Hyperlink graph

$$\bar{A} = \begin{pmatrix} 0 & 1/2 & 1/2 & 0 & 0 & 0 \\ 1/2 & 0 & 1/2 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1/3 & 0 & 1/3 & 1/3 \\ 1/6 & 1/6 & 1/6 & 1/6 & 1/6 & 1/6 \\ 0 & 0 & 0 & 1/2 & 1/2 & 0 \end{pmatrix}.$$

Transition probability matrix

Modification II: make $\bar{A}$ **irreducible** and **aperiodic**

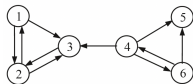
Add a link from each page to every page and give each such link a small transition probability controlled by a parameter d .



$$\bar{A} = \begin{pmatrix} 0 & 1/2 & 1/2 & 0 & 0 & 0 \\ 1/2 & 0 & 1/2 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1/3 & 0 & 1/3 & 1/3 \\ 1/6 & 1/6 & 1/6 & 1/6 & 1/6 & 1/6 \\ 0 & 0 & 0 & 1/2 & 1/2 & 0 \end{pmatrix}$$

Modification II: make $\bar{A}$ **irreducible** and **aperiodic**

Add a link from each page to every page and give each such link a small transition probability controlled by a parameter d .



$$(1-d)\frac{\bar{E}}{n} + d\bar{A}^T = \begin{pmatrix} 1/60 & 7/15 & 1/60 & 1/60 & 1/6 & 1/60 \\ 7/15 & 1/60 & 11/12 & 1/60 & 1/6 & 1/60 \\ 7/15 & 7/15 & 1/60 & 19/60 & 1/6 & 1/60 \\ 1/60 & 1/60 & 1/60 & 1/60 & 1/6 & 7/15 \\ 1/60 & 1/60 & 1/60 & 19/60 & 1/6 & 7/15 \\ 1/60 & 1/60 & 1/60 & 19/60 & 1/6 & 1/60 \end{pmatrix}$$

$(1-d)\frac{\bar{E}}{n} + d\bar{A}^T$ is **stochastic** (but transposed), **irreducible**, and **aperiodic**

Apply the power iteration algorithm to the modified $\bar{A}$

We can now apply the **power iteration algorithm** to the modified matrix (that now satisfies the 3 conditions!).

The algorithm will compute the **principal** eigenvector for the matrix.

The resulting vector is the PageRank vector $\bar{P}$. This leads to the PageRank algorithm.

PageRank algorithm

PageRank (An n -vertex graph without dangling vertices: $G = (V, E)$; damping factor: d ; tolerance ϵ)

- 1: $P_0(i) = 1/n$ for all $i = 1, \dots, n$; // initial probability distribution (i.e. initial PageRank values)
- 2: $k = 1$
- 2: repeat
- 3: $P_k(i) = \frac{1-d}{n} + d \cdot \sum_{(x,i) \in E} \frac{P_{k-1}(x)}{O_x}$, for all $i = 1, \dots, n$; // update PageRank values
- 4: $k = k + 1$
- 5: until $\|\bar{P}_k - \bar{P}_{k-1}\| \leq \epsilon$
- 6: return $\bar{P}_k = (P_k(1), P_k(2), \dots, P_k(n))$ // the final PageRank values

Remarks

- ▶ In practice, a typical value for the damping factor d is 0.85
- ▶ PageRank algorithm can be applied to any graph, not limiting to web graph, to induce a ranking for the vertices
- ▶ PageRank algorithm is one of many algorithms that are based on the idea of random walks in a graph
- ▶ Since we are only interested in the ranking of the pages, the actual convergence may not be necessary, and the algorithm can be terminated after some maximum number of iterations even before achieving the tolerance ϵ
- ▶ In [1], it is reported that on a database of **322 million links** the algorithm converges to an acceptable tolerance in roughly **52 iterations**.

[1] Brin, S. and P. Lawrence. The anatomy of a large-scale hypertextual web search engine. Computer Networks, 1998, 30(1–7): p. 107–117.